



A Project Report on

FINITE ELEMENT ANALYSIS OF COMPOSITE CANTILEVER BEAM USING MATLAB & ANSYS

Finite Element Analysis (ME60407)

Under the Guidance of

Prof. Sanjay Gupta

Department of Mechanical Engineering, IIT Kharagpur

Year 2025-2026

Indian institute of Technology Kharagpur, 721302

Submitted by

Nikkula Krishnakarthik(22MF3IM29)

Abstract

This project details the development and validation of a MATLAB Finite Element Analysis (FEA) code for a Cantilever beam constructed from three distinct materials. A fine mesh was created, capturing variations in material properties across the beam's dimensions, and enabling calculation of field variables such as displacement, Strain, and Stress.

Verification and validation were conducted by comparing MATLAB outputs to ANSYS simulation results, focusing on nodes at material connections and at the point of maximum displacement. The results confirmed the MATLAB code's accuracy and applicability in multi-material beam analysis, supporting its use for reliable, detailed structural modeling under transverse loading conditions.

Keywords: Finite Element Analysis, MATLAB, ANSYS APDL, Verification, Validation, Meshing, Simulation.

Acknowledgement

I would like to express my sincere gratitude to **Prof. Sanjay Gupta** for his invaluable guidance, encouragement, and expertise throughout this project. His insights and support were instrumental in the successful development and validation of the Finite Element Analysis (FEA) code.

I also extend my thanks to the **Department of Mechanical Engineering at IIT Kharagpur** for providing the resources and an excellent research environment that enabled the completion of this work.

At last, I would like to thank my colleagues and seniors for their guidance and support during the course of this project. Their assistance and feedback greatly contributed to the quality and success of this work.

Nikkula Krishnakarthik(22MF3IM29)

Sign:

Date: November 3, 2025

Place: IIT Kharagpur

Contents

Abstract	1
Acknowledgement	2
1 Problem Statement	4
2 Aim and Objectives	6
2.1 Aim	6
2.2 Objectives	6
3 Formulation	7
3.1 Element Selection and Shape Functions	7
3.2 Strain-Displacement and Stress-Strain	7
3.3 Element and Global Stiffness Matrix	7
3.4 Solving the System	8
4 Flow Chart	9
5 MATLAB Code	10
6 ANSYS APDL Code	15
7 Comparison of MATLAB & ANSYS Result	17
7.1 Comparison of Displacement in Vertical Direction (UY)	18
7.2 Comparison of Displacement in Horizontal Direction (UX)	19
7.3 Comparison of Normal Stress in Horizontal Direction (σ_x)	20
7.4 Comparison of Normal Stress in Vertical Direction (σ_y)	21
7.5 Comparison of Shear Stress (τ_{xy})	22
7.6 Comparison of Von Mises Stress	23
8 Conclusion & References	24
8.1 Conclusion	24
8.2 References	24

1 Problem Statement

Write a MATLAB code for the given beam made by three different materials, fixed at one end and subjected to a transverse load of 500 N at the free end, using Finite Element Analysis.

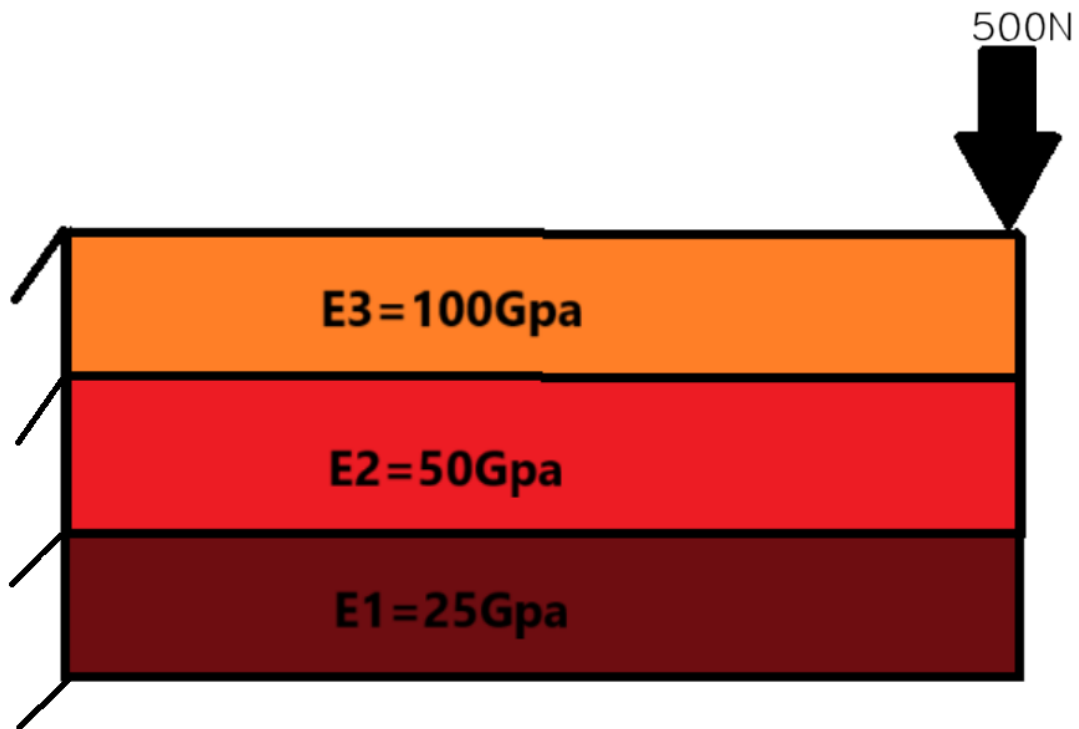


Figure 1: Problem Definition

Dimensions:

- Length = 95 mm.
- Total Height = 48 mm (16 mm per layer).
- Thickness = 1 mm.

Material Properties:

- $E_1 = 25\text{ GPa}$ (Bottom Layer)
- $E_2 = 50\text{ GPa}$ (Middle Layer)
- $E_3 = 100\text{ GPa}$ (Top Layer)
- $\nu = 0.3$ (Poisson's ratio, uniform for all layers)

Load and Boundary Conditions:

- A total transverse load of 500 N is applied at the free end ($x = 95\text{ mm}$), distributed evenly among the nodes.

- The beam is fixed at the left end ($x = 0$).

Assumption:

- Since the thickness (1 mm) is very small compared to the length and height, a **Plane Stress** condition is assumed.

2 Aim and Objectives

2.1 Aim

To develop and validate a MATLAB-based Finite Element Analysis (FEA) code for accurately modeling the mechanical behavior of a multi-material cantilever beam under transverse loading.

2.2 Objectives

- To create a MATLAB script using 4-node quadrilateral elements to mesh the beam and calculate the following field variables:
 - Nodal Displacements (U_x, U_y)
 - Element Strains ($\epsilon_x, \epsilon_y, \gamma_{xy}$)
 - Element Stresses ($\sigma_x, \sigma_y, \tau_{xy}$)
- To model the identical problem in ANSYS APDL to obtain a benchmark solution.
- To verify and validate the custom MATLAB code by comparing its results (contour plots, interface nodal values, and extreme values) against the ANSYS simulation.

3 Formulation

To solve this 2D plane stress problem, the following Finite Element formulation was used.

3.1 Element Selection and Shape Functions

A 4-node bilinear quadrilateral element (Q4) is used. The displacement at any point (ξ, η) inside the element is interpolated from the nodal displacements $\{u_e\}$ using shape functions N_i :

$$u(\xi, \eta) = \sum_{i=1}^4 N_i(\xi, \eta) u_i$$

$$v(\xi, \eta) = \sum_{i=1}^4 N_i(\xi, \eta) v_i$$

The shape functions in natural coordinates (ξ, η) are:

$$N_1 = \frac{1}{4}(1 - \xi)(1 - \eta) \quad N_2 = \frac{1}{4}(1 + \xi)(1 - \eta)$$

$$N_3 = \frac{1}{4}(1 + \xi)(1 + \eta) \quad N_4 = \frac{1}{4}(1 - \xi)(1 + \eta)$$

3.2 Strain-Displacement and Stress-Strain

The strain-displacement matrix $[B]$ relates the element strains $\{\epsilon\}$ to the nodal displacements $\{u_e\}$:

$$\{\epsilon\} = [B]\{u_e\}$$

where $\{\epsilon\} = \{\epsilon_x, \epsilon_y, \gamma_{xy}\}^T$ and $\{u_e\} = \{u_1, v_1, \dots, u_4, v_4\}^T$.

The stress-strain relationship for 2D plane stress is:

$$\{\sigma\} = [D]\{\epsilon\}$$

where $[D]$ is the material elasticity matrix:

$$[D] = \frac{E}{1 - \nu^2} \begin{bmatrix} 1 & \nu & 0 \\ \nu & 1 & 0 \\ 0 & 0 & \frac{1-\nu}{2} \end{bmatrix}$$

Note that the Young's Modulus, E , will be E_1 , E_2 , or E_3 depending on the element's location.

3.3 Element and Global Stiffness Matrix

The element stiffness matrix $[K_e]$ is calculated using 2x2 Gauss Quadrature:

$$[K_e] = \int_V [B]^T [D] [B] dV = t \int_{-1}^1 \int_{-1}^1 [B]^T [D] [B] \det[J] d\xi d\eta$$

The global stiffness matrix $[K]$ is assembled by summing the contributions from all element stiffness matrices.

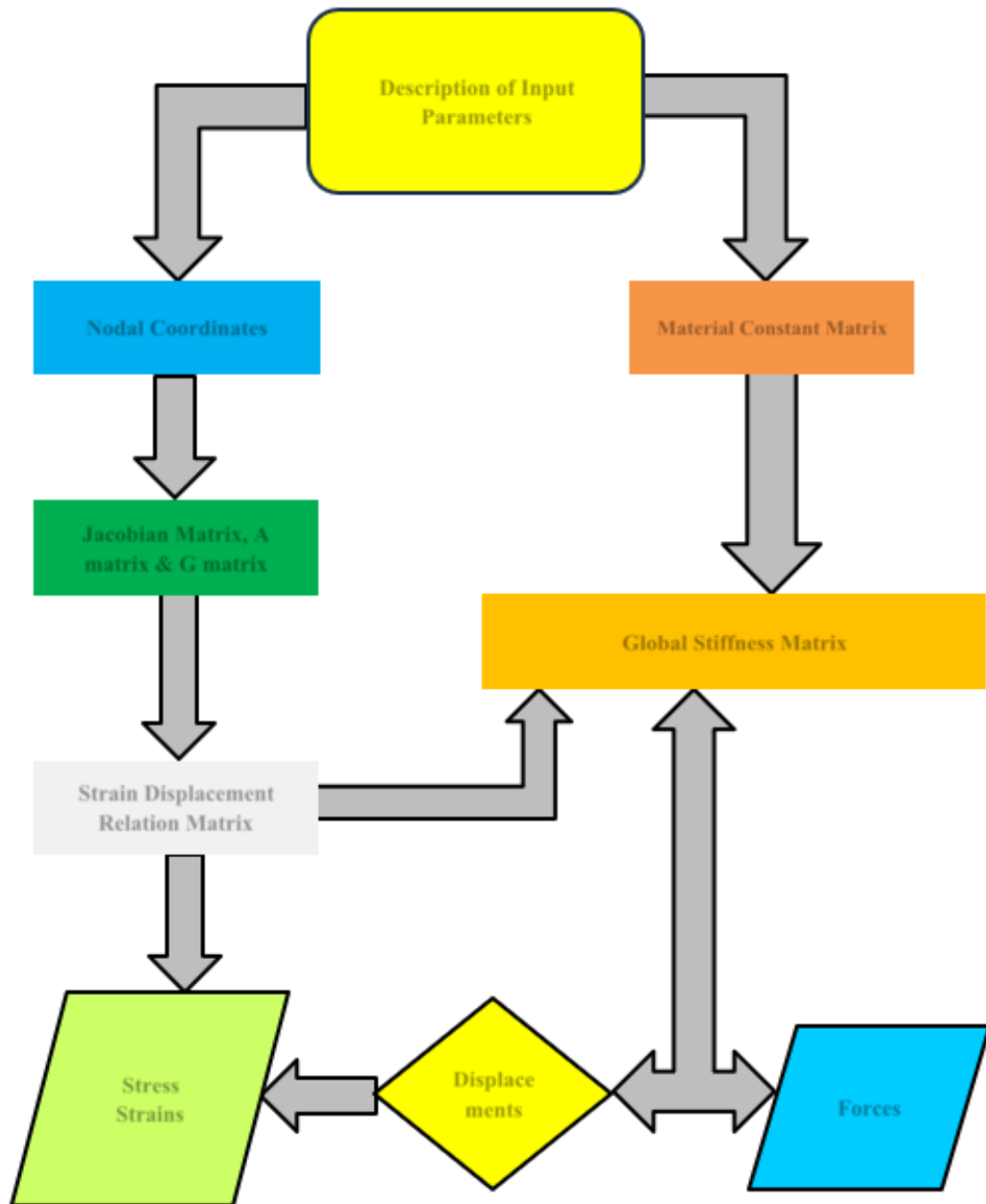
3.4 Solving the System

The global system of equations is assembled and solved for the unknown nodal displacements $\{U\}$:

$$[K]\{U\} = \{F\}$$

Boundary conditions (zero displacement) are applied at the fixed end ($x = 0$) by modifying the $[K]$ matrix and $\{F\}$ vector. The load vector $\{F\}$ contains the nodal forces applied at the free end.

4 Flow Chart



5 MATLAB Code

This is the 'FEM.m' script I used for the analysis.

```
1 %% Finite Element Analysis of Composite Beam with Three Materials
2 % Problem: Beam with three materials subjected to transverse
   load of 500N
3 clear all; clc; close all;
4
5 %% Material Properties
6 E1 = 25e9;      % Young's modulus for material 1 (Pa)
7 E2 = 50e9;      % Young's modulus for material 2 (Pa)
8 E3 = 100e9;     % Young's modulus for material 3 (Pa)
9 nu = 0.3;       % Poisson's ratio (uniform)
10 thickness = 1e-3; % Beam thickness (m)
11
12 %% Geometry Parameters (converting mm to m)
13 L = 95e-3;      % Total length (m)
14 h1 = 16e-3;     % Height of segment 1 (m)
15 h2 = 16e-3;     % Height of segment 2 (m)
16 h3 = 16e-3;     % Height of segment 3 (m)
17 total_height = h1 + h2 + h3; % Total height = 48mm
18
19 %% Applied Load
20 F = 500; % Transverse load (N)
21
22 %% Node Generation
23 num_elements_length = 20;
24 num_elements_height_per_section = 4;
25 num_nodes_x = num_elements_length + 1;
26 num_nodes_y_per_section = num_elements_height_per_section + 1;
27
28 node_id = 1;
29 nodes = [];
30
31 x_coords = linspace(0, L, num_nodes_x);
32 y1_coords = linspace(0, h1, num_nodes_y_per_section);
33 y2_coords = linspace(h1, h1+h2, num_nodes_y_per_section);
34 y3_coords = linspace(h1+h2, total_height,
   num_nodes_y_per_section);
35
36 y_coords = [y1_coords, y2_coords(2:end), y3_coords(2:end)];
37 num_nodes_y = length(y_coords);
38
39 for j = 1:num_nodes_y
40     for i = 1:num_nodes_x
41         nodes(node_id, :) = [x_coords(i), y_coords(j)];
42         node_id = node_id + 1;
43     end
44 end
45 num_nodes = size(nodes, 1);
46
```

```

47 %% Element Generation
48 element_id = 1;
49 elements = [];
50 material_ids = [];
51
52 for j = 1:(num_nodes_y-1)
53     if y_coords(j) < h1
54         material = 1; % E1
55     elseif y_coords(j) < (h1 + h2)
56         material = 2; % E2
57     else
58         material = 3; % E3
59     end
60
61     for i = 1:(num_nodes_x-1)
62         n1 = (j-1)*num_nodes_x + i;
63         n2 = (j-1)*num_nodes_x + i + 1;
64         n3 = j*num_nodes_x + i + 1;
65         n4 = j*num_nodes_x + i;
66
67         elements(element_id, :) = [n1, n2, n3, n4];
68         material_ids(element_id) = material;
69         element_id = element_id + 1;
70     end
71 end
72 num_elements = size(elements, 1);
73
74 %% Global Stiffness Matrix Assembly
75 dof_per_node = 2;
76 total_dof = num_nodes * dof_per_node;
77 K_global = zeros(total_dof, total_dof);
78 F_global = zeros(total_dof, 1);
79
80 %% Element Stiffness Matrix Calculation
81 for elem = 1:num_elements
82     elem_nodes = elements(elem, :);
83     xe = nodes(elem_nodes, 1);
84     ye = nodes(elem_nodes, 2);
85
86     material = material_ids(elem);
87     if material == 1
88         E = E1;
89     elseif material == 2
90         E = E2;
91     else
92         E = E3;
93     end
94
95     D = (E/(1-nu^2)) * [1, nu, 0;
96                        nu, 1, 0;
97                        0, 0, (1-nu)/2];

```

```

98
99 Ke = zeros(8, 8);
100 gp = [-1/sqrt(3), 1/sqrt(3)];
101
102 for i = 1:2
103     for j = 1:2
104         xi = gp(i);
105         eta = gp(j);
106
107         dN_dxi = 0.25*[-(1-eta); (1-eta); (1+eta); -(1+eta)];
108         dN_deta = 0.25*[-(1-xi); -(1+xi); (1+xi); (1-xi)];
109
110         J = [dN_dxi'; dN_deta'] * [xe, ye];
111         detJ = det(J);
112         J_inv = inv(J);
113
114         dN = J_inv * [dN_dxi'; dN_deta'];
115
116         B = zeros(3, 8);
117         for k = 1:4
118             B(1, 2*k-1) = dN(1, k);
119             B(2, 2*k) = dN(2, k);
120             B(3, 2*k-1) = dN(2, k);
121             B(3, 2*k) = dN(1, k);
122         end
123
124         Ke = Ke + B' * D * B * detJ * thickness;
125     end
126 end
127
128 dof_elem = zeros(1, 8);
129 for k = 1:4
130     dof_elem(2*k-1) = 2*elem_nodes(k) - 1;
131     dof_elem(2*k) = 2*elem_nodes(k);
132 end
133
134 K_global(dof_elem, dof_elem) = K_global(dof_elem, dof_elem)
    + Ke;
135 end
136
137 %% Apply Boundary Conditions
138 fixed_nodes = find(nodes(:, 1) == 0);
139 fixed_dofs = [2*fixed_nodes-1; 2*fixed_nodes];
140
141 %% Apply Load
142 loaded_nodes = find(nodes(:, 1) == L);
143 load_per_node = F / length(loaded_nodes);
144
145 for node = loaded_nodes'
146     F_global(2*node) = F_global(2*node) - load_per_node;
147 end

```

```

148
149 %% Solve System
150 all_dofs = 1:total_dof;
151 free_dofs = setdiff(all_dofs, fixed_dofs);
152
153 U = zeros(total_dof, 1);
154 U(free_dofs) = K_global(free_dofs, free_dofs) \
    F_global(free_dofs);
155
156 %% Post-Processing
157 u_disp = U(1:2:end);
158 v_disp = U(2:2:end);
159 nodes_deformed = nodes + [u_disp, v_disp];
160
161 %% Calculate Stresses at element centers
162 stress_xx = zeros(num_elements, 1);
163 stress_yy = zeros(num_elements, 1);
164 stress_xy = zeros(num_elements, 1);
165 von_mises = zeros(num_elements, 1);
166
167 for elem = 1:num_elements
168     elem_nodes = elements(elem, :);
169     xe = nodes(elem_nodes, 1);
170     ye = nodes(elem_nodes, 2);
171
172     ue = zeros(8, 1);
173     for k = 1:4
174         ue(2*k-1) = U(2*elem_nodes(k)-1);
175         ue(2*k) = U(2*elem_nodes(k));
176     end
177
178     material = material_ids(elem);
179     if material == 1
180         E = E1;
181     elseif material == 2
182         E = E2;
183     else
184         E = E3;
185     end
186
187     D = (E/(1-nu^2)) * [1, nu, 0; nu, 1, 0; 0, 0, (1-nu)/2];
188
189     % Calculate at element center (xi=0, eta=0)
190     xi = 0; eta = 0;
191     dN_dxi = 0.25*[-(1-eta); (1-eta); (1+eta); -(1+eta)];
192     dN_deta = 0.25*[-(1-xi); -(1+xi); (1+xi); (1-xi)];
193
194     J = [dN_dxi'; dN_deta'] * [xe, ye];
195     J_inv = inv(J);
196     dN = J_inv * [dN_dxi'; dN_deta'];
197

```

```

198 B = zeros(3, 8);
199 for k = 1:4
200     B(1, 2*k-1) = dN(1, k);
201     B(2, 2*k) = dN(2, k);
202     B(3, 2*k-1) = dN(2, k);
203     B(3, 2*k) = dN(1, k);
204 end
205
206 strain = B * ue;
207 stress = D * strain;
208
209 stress_xx(elem) = stress(1);
210 stress_yy(elem) = stress(2);
211 stress_xy(elem) = stress(3);
212
213 von_mises(elem) = sqrt(stress_xx(elem)^2 - ...
214     stress_xx(elem)*stress_yy(elem) + stress_yy(elem)^2 +
215     3*stress_xy(elem)^2);
216 end

```

Listing 1: MATLAB Code for 3-Material Beam

6 ANSYS APDL Code

The following APDL script was used to create the validation model in ANSYS. The script uses SI units (meters, Pascals, Newtons) to match the MATLAB code.

```
1 FINISH
2 /CLEAR
3 /TITLE, Finite Element Analysis of 3-Material Composite Beam
4 ! Units: SI (m, N, Pa, kg)
5
6 ! --- 1. Setup ---
7 /PREP7                                ! Enter Preprocessor
8 ANTYPE, STATIC
9 ET, 1, PLANE182
10 KEYOPT, 1, 3, 3
11 R, 1, 1e-3
12
13 ! --- Material Properties (SI Units) ---
14 MP, EX, 1, 25e9                        ! E = 25 GPa
15 MP, NUXY, 1, 0.3
16 MP, EX, 2, 50e9                        ! E = 50 GPa
17 MP, NUXY, 2, 0.3
18 MP, EX, 3, 100e9                       ! E = 100 GPa
19 MP, NUXY, 3, 0.3
20
21 ! --- 2. Geometry (Top-Down Modeling) ---
22 L = 95e-3
23 h1 = 16e-3
24 h2 = 16e-3
25 h3 = 16e-3
26 h_total = h1 + h2 + h3
27 A1 = RECTNG(0, L, 0, h1)
28 A2 = RECTNG(0, L, h1, h1+h2)
29 A3 = RECTNG(0, L, h1+h2, h_total)
30 AGLUE, ALL
31
32 ! --- 3. Meshing (Matches MATLAB) ---
33 MSHAPE, 0, 2D
34 MSHKEY, 1
35 LSEL, S, LOC, Y, 0
36 LSEL, A, LOC, Y, h1
37 LSEL, A, LOC, Y, h1+h2
38 LSEL, A, LOC, Y, h_total
39 LESIZE, ALL, , , 20                    ! 20 divisions
40 ALLSEL, ALL
41 LSEL, S, LOC, X, 0
42 LSEL, A, LOC, X, L
43 LESIZE, ALL, , , 4                      ! 4 divisions (per section)
44 ALLSEL, ALL
45 AATT, 1, 1, 1
46 ASEL, S, AREA, , A2
47 AATT, 2, 1, 1
```



```

48 ASEL, S, AREA, , A3
49 AATT, 3, 1, 1
50 ALLSEL, ALL
51 AMESH, ALL
52
53 ! --- 4. Boundary Conditions & Loads ---
54 FINISH
55 /SOLU
56 LSEL, S, LOC, X, 0
57 DL, ALL, , ALL, 0
58 ALLSEL, ALL
59 LSEL, S, LOC, X, L
60 NSEL, S, ON, LINES
61 F_TOTAL = -500
62 *GET, NCOUNT, NODE, 0, COUNT
63 F_PER_NODE = F_TOTAL / NCOUNT
64 F, ALL, FY, F_PER_NODE
65 ALLSEL, ALL
66 SOLVE
67 FINISH
68
69 ! --- 5. Post-Processing ---
70 /POST1
71 PLDISP, 1
72 ! --- Plot Stress Distributions (in Pa) ---
73 PLNSOL, S, EQV, 0, 1
74 /TITLE, Von Mises Stress (Pa)
75 PLNSOL, S, X, 0, 1
76 /TITLE, Normal Stress sigma_xx (Pa)
77 PLNSOL, S, Y, 0, 1
78 /TITLE, Normal Stress sigma_yy (Pa)
79 PLNSOL, S, XY, 0, 1
80 /TITLE, Shear Stress tau_xy (Pa)

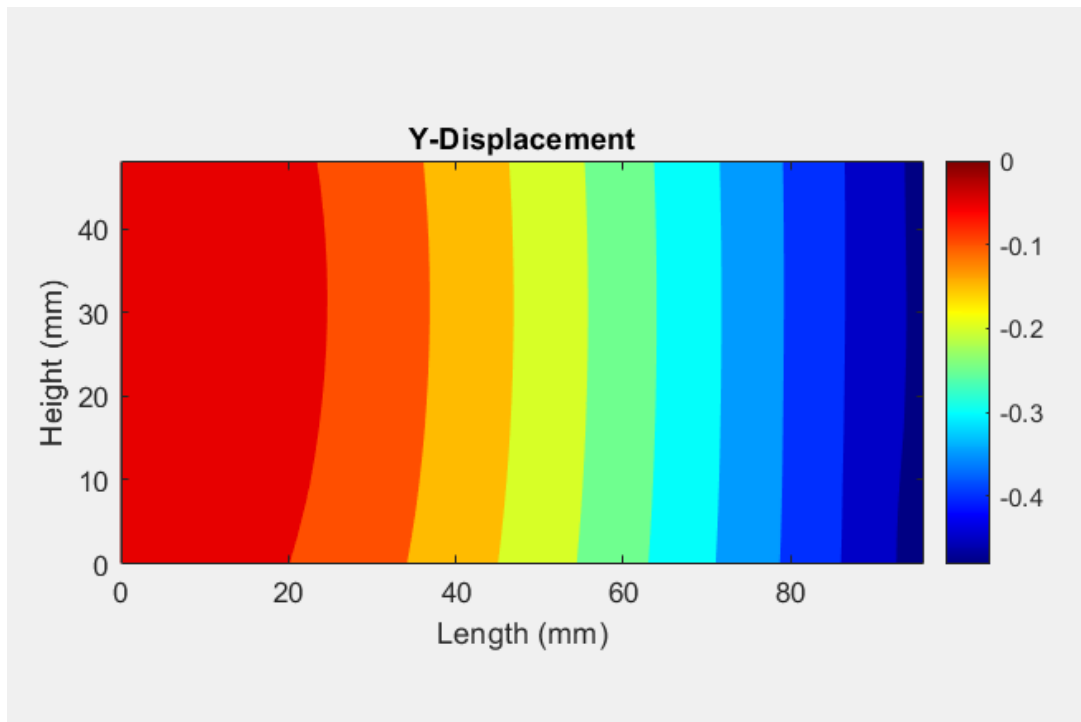
```

Listing 2: ANSYS APDL Code for Validation

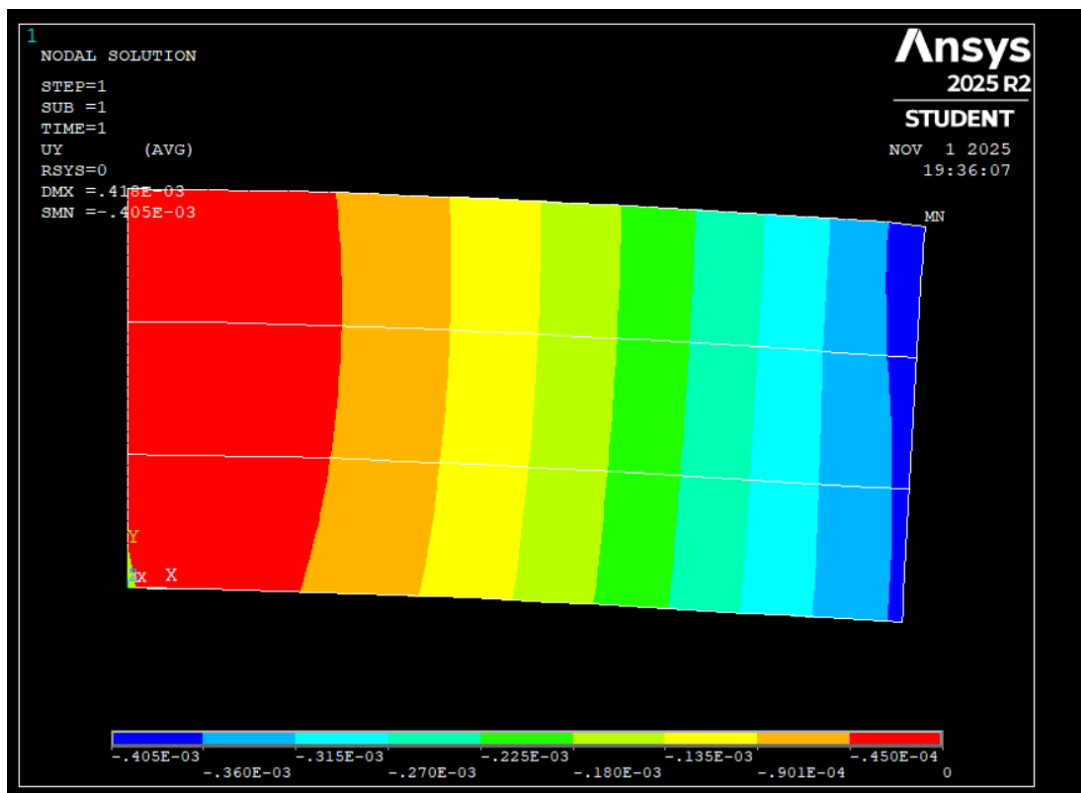
7 Comparison of MATLAB & ANSYS Result

The MATLAB results are compared with ANSYS APDL results to evaluate the accuracy and consistency of the simulation under identical conditions.

7.1 Comparison of Displacement in Vertical Direction (UY)



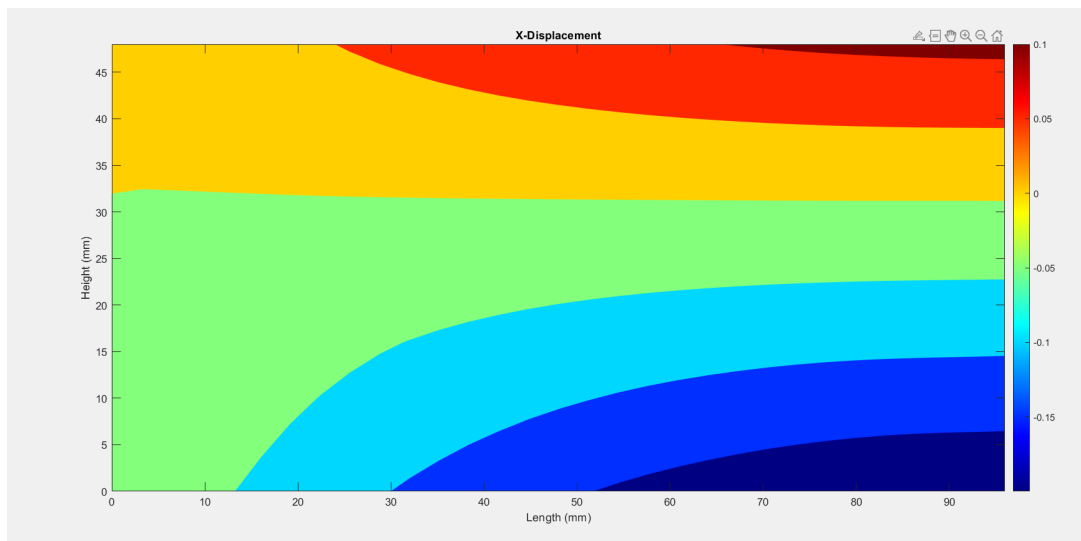
(a) MATLAB Result (mm)



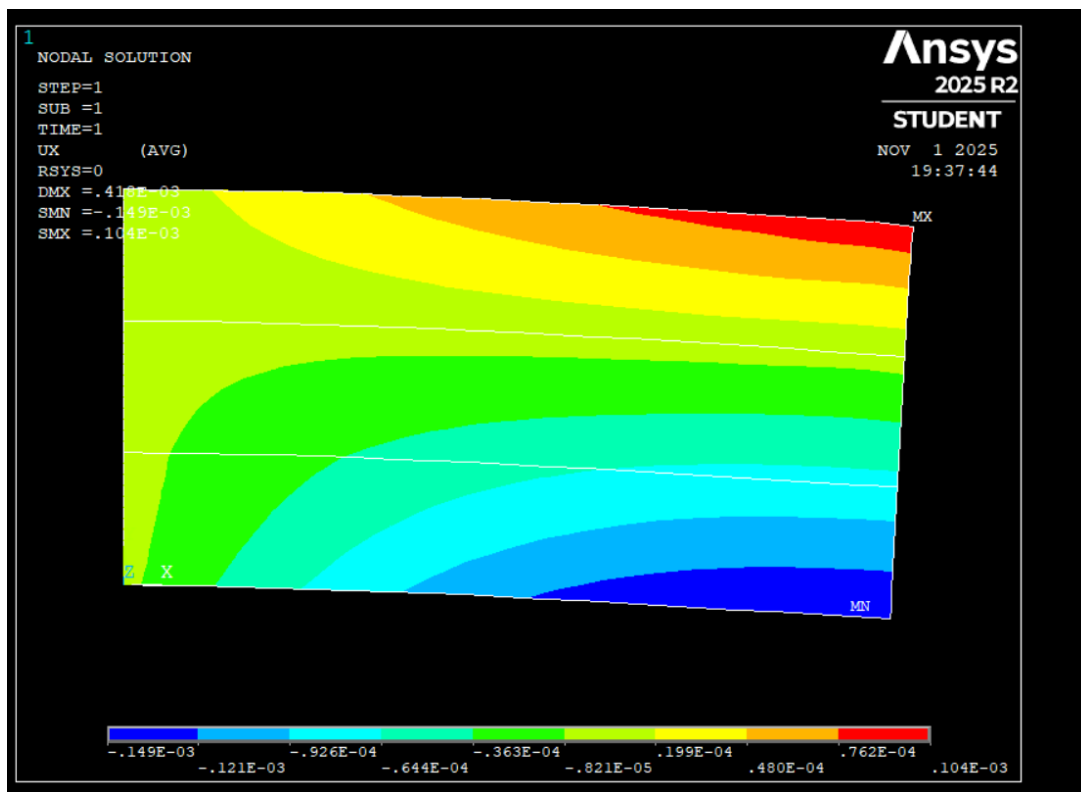
(b) ANSYS APDL Result (m)

Figure 2: Comparison of Vertical Displacement (UY)

7.2 Comparison of Displacement in Horizontal Direction (UX)



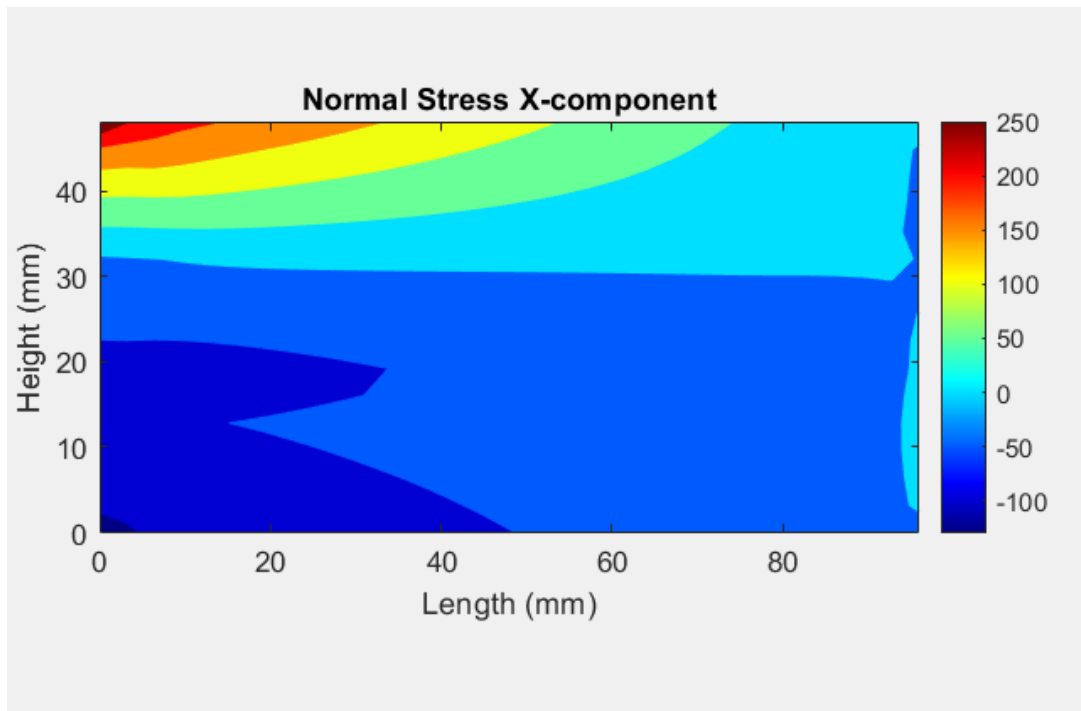
(a) MATLAB Result (mm)



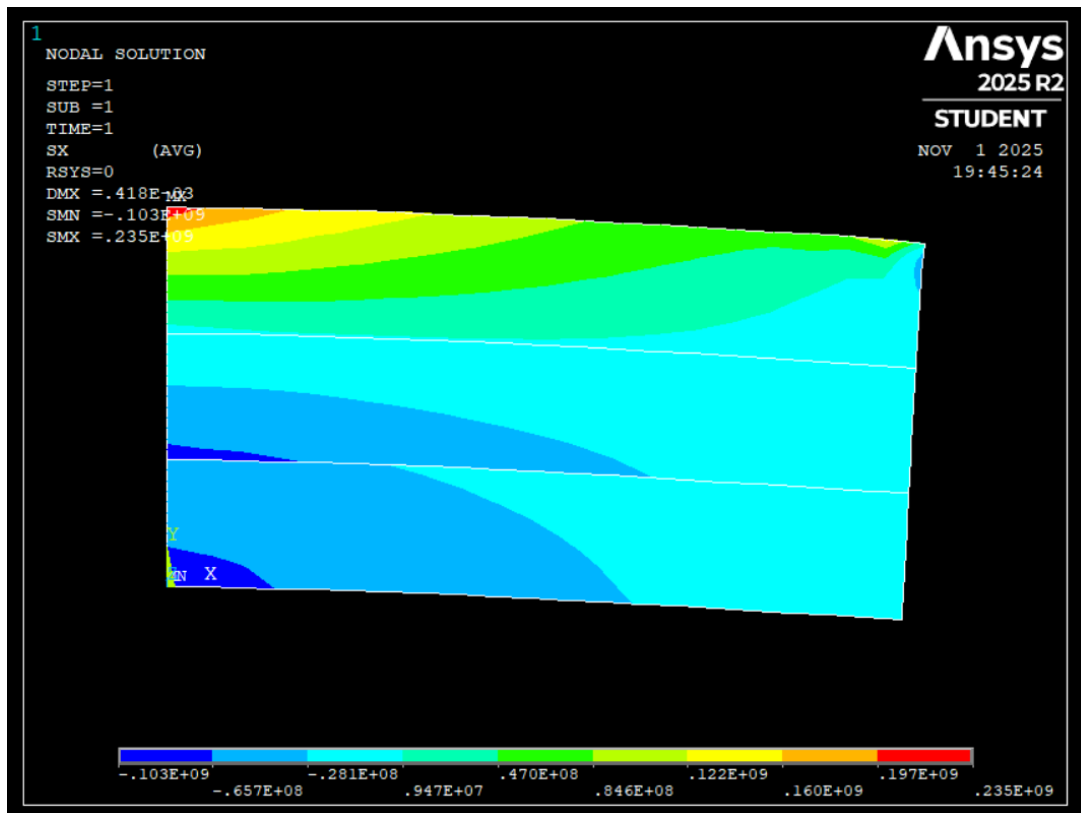
(b) ANSYS APDL Result (m)

Figure 3: Comparison of Horizontal Displacement (UX)

7.3 Comparison of Normal Stress in Horizontal Direction (σ_x)



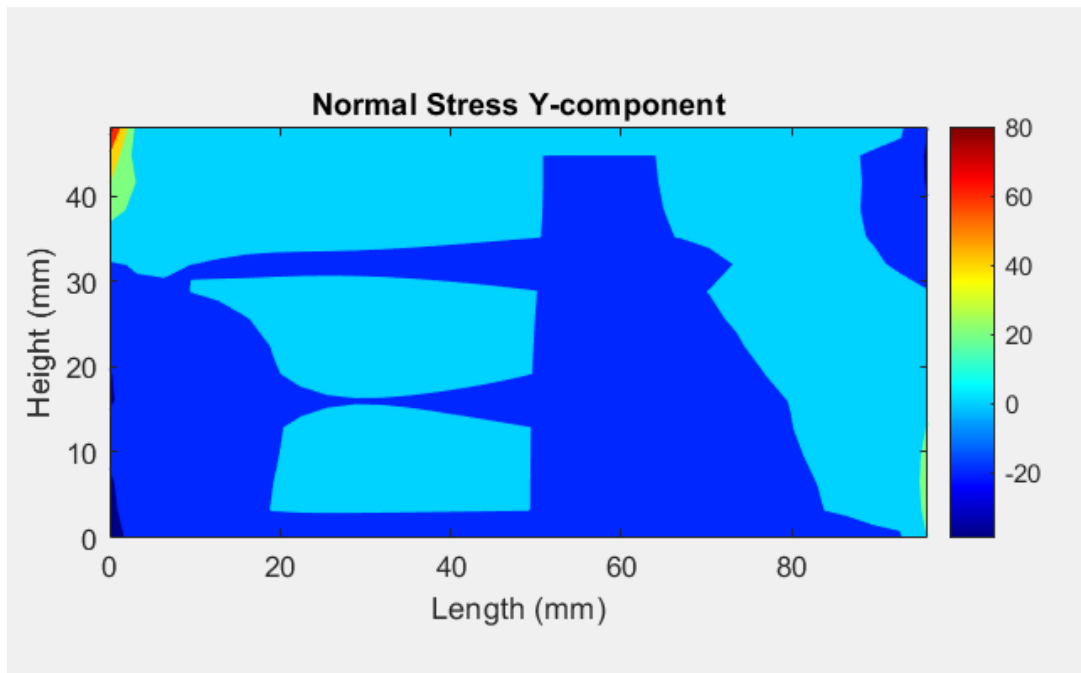
(a) MATLAB Result (MPa)



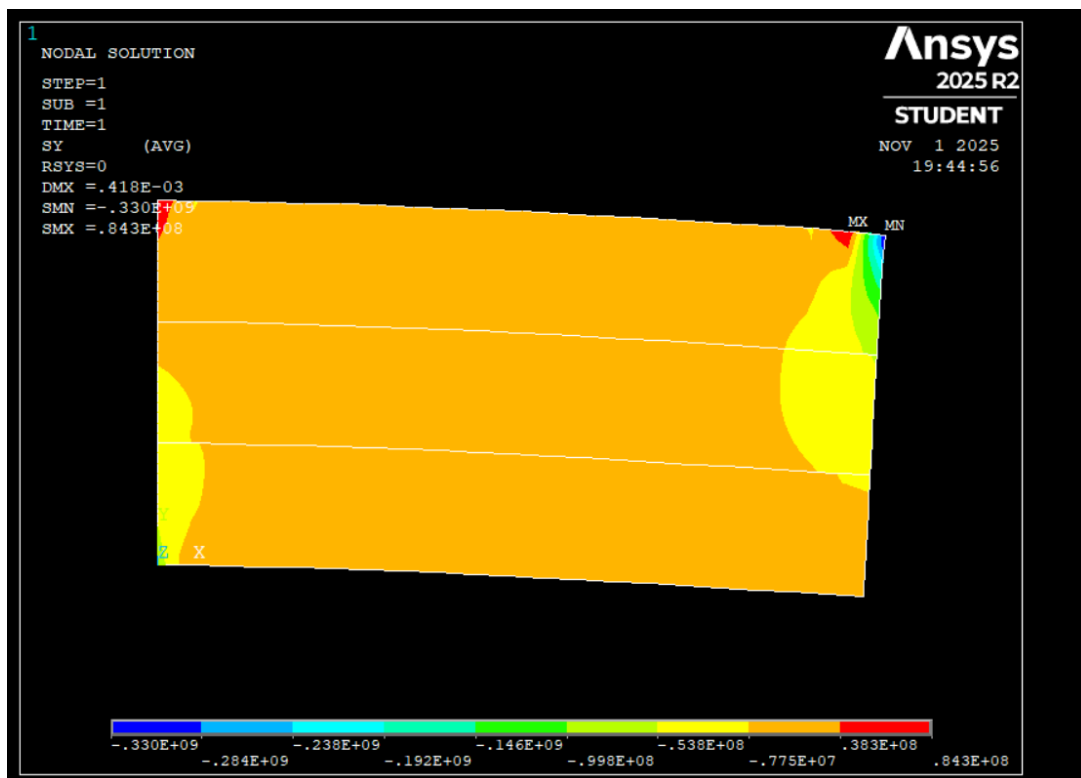
(b) ANSYS APDL Result (Pa)

Figure 4: Comparison of Normal Stress (σ_x)

7.4 Comparison of Normal Stress in Vertical Direction (σ_y)



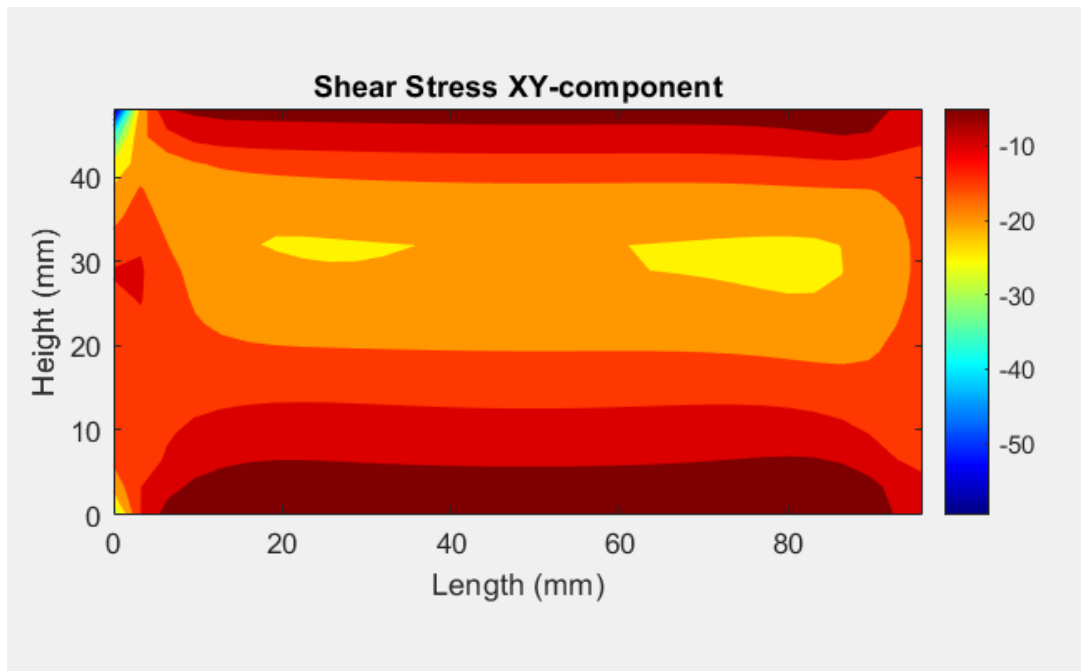
(a) MATLAB Result (MPa)



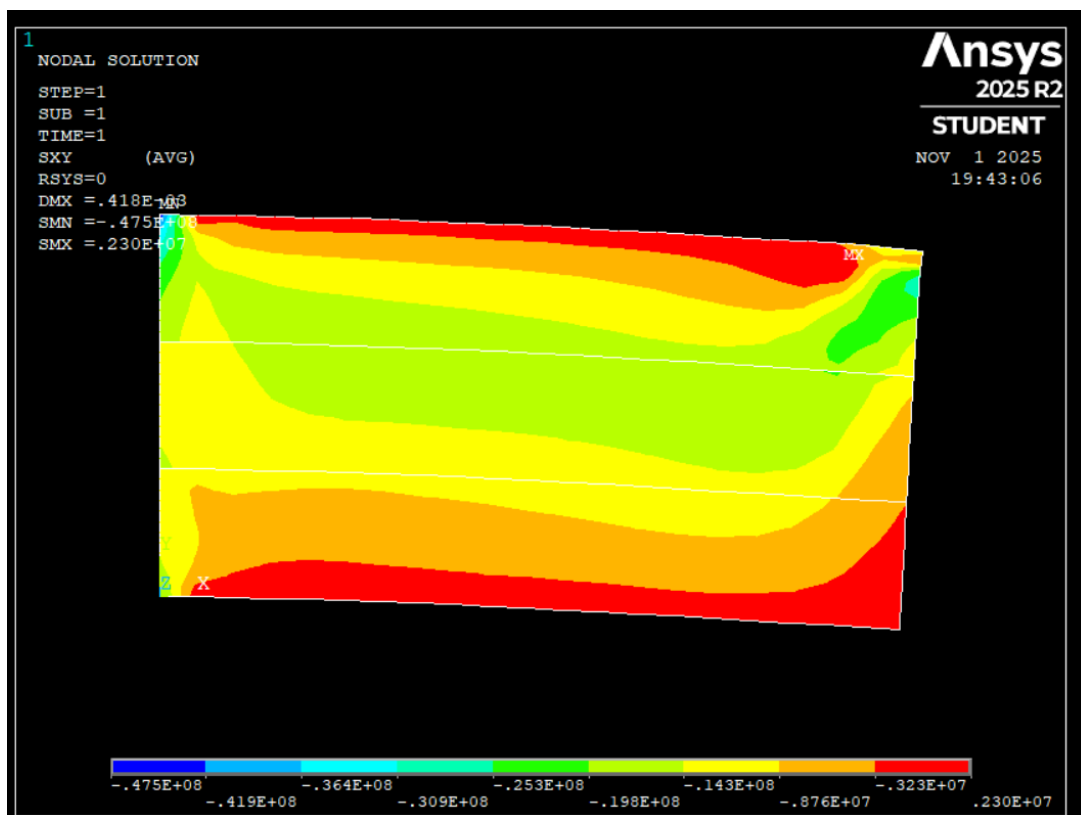
(b) ANSYS APDL Result (Pa)

Figure 5: Comparison of Normal Stress (σ_y)

7.5 Comparison of Shear Stress (τ_{xy})



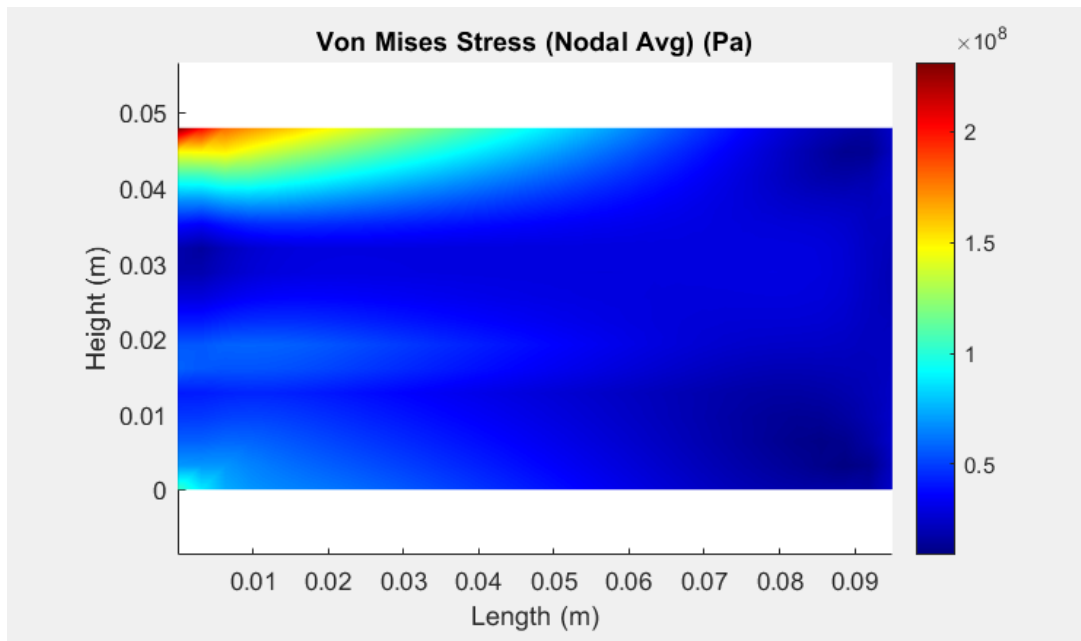
(a) MATLAB Result (MPa)



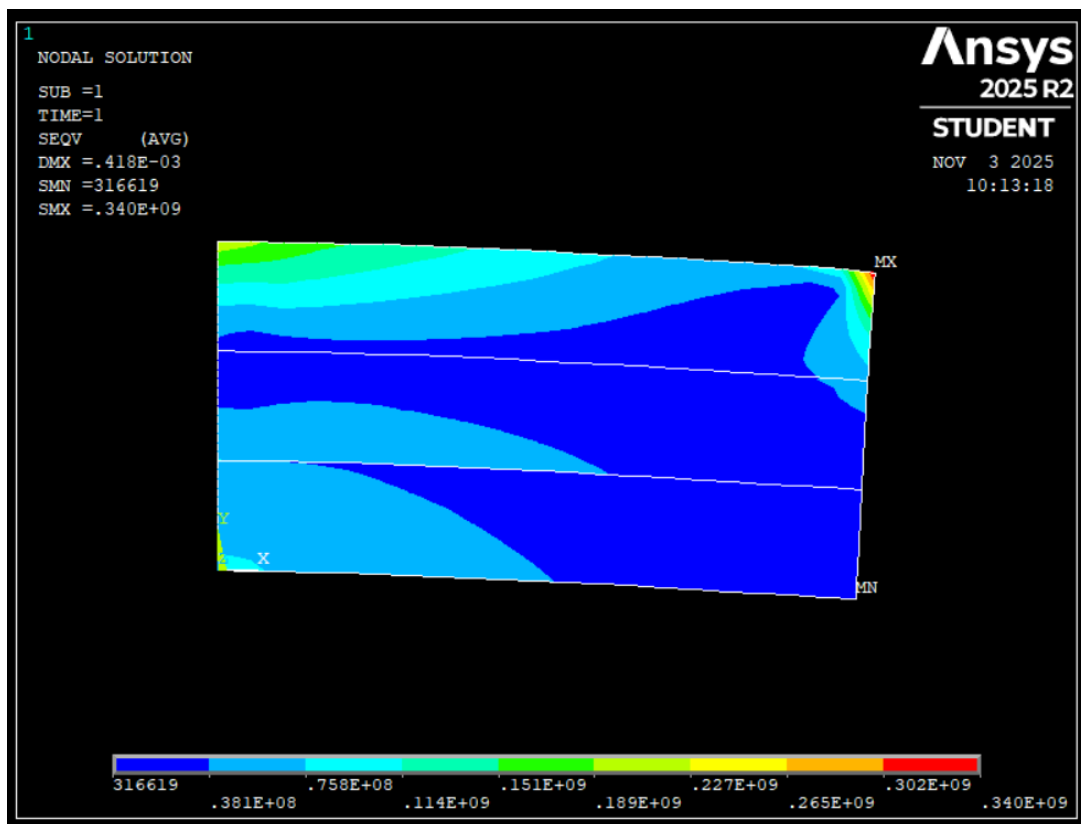
(b) ANSYS APDL Result (Pa)

Figure 6: Comparison of Shear Stress (τ_{xy})

7.6 Comparison of Von Mises Stress



(a) MATLAB Result (MPa)



(b) ANSYS APDL Result (Pa)

Figure 7: Comparison of Von Mises Stress

8 Conclusion & References

8.1 Conclusion

The MATLAB-based Finite Element Analysis (FEA) code in this project provided accurate results for a multi-material beam under transverse loading. The results are in strong agreement with those obtained from ANSYS Mechanical APDL. This validation confirms that the finite element formulation and the MATLAB code implemented are reliable for capturing key field variables such as displacement and stress distribution in complex, multi-material structures.

Increasing the number of elements in the MATLAB model could enhance the precision of results, and a convergence test could establish the optimal mesh density, achieving stability in results and ensuring computational efficiency.

8.2 References

References

- [1] Entwistle, K. M. (2001). *Basic Principles of the Finite Element Method*. Maney.
- [2] Ferreira, A. J. M. (2008). *MATLAB Codes for Finite Element Analysis*. Springer.
- [3] Reddy, J. N. (2006). *An Introduction to the Finite Element Method*. McGraw-Hill.